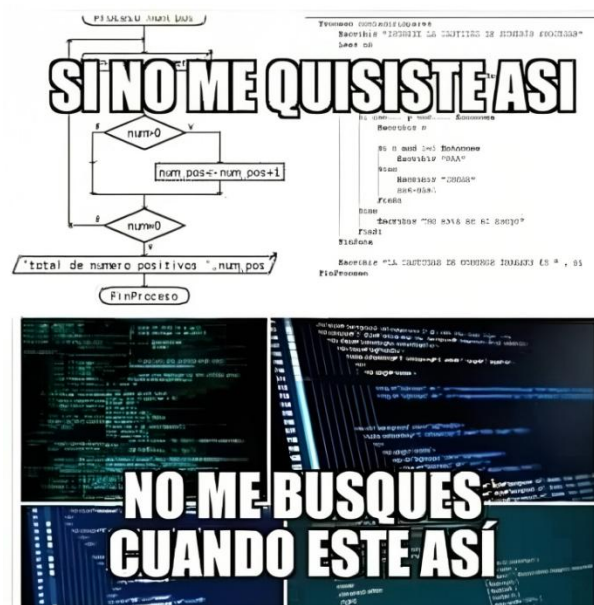




Marco Teórico

Descripción técnica detallada

La arquitectura Space-Based (SBA) representa una solución altamente eficaz para sistemas distribuidos que enfrentan demandas extremas de escalabilidad y resiliencia. Su diseño responde a los desafíos contemporáneos de procesamiento masivo, alta concurrencia y tolerancia a fallos, especialmente en entornos donde los picos de carga pueden comprometer la estabilidad de arquitecturas tradicionales.

Su principio clave es:

- ❖ Eliminación de cuellos de botella: SBA evita la dependencia de bases de datos centralizadas, que suelen convertirse en puntos críticos de fallo o saturación. Al distribuir los datos en un espacio compartido en memoria, se reduce drásticamente la latencia y se mejora el rendimiento global del sistema.
- ❖ Escalabilidad horizontal real: Las *Processing Units (PUs)* permiten escalar el sistema simplemente añadiendo más nodos. Cada PU contiene lógica de negocio y caché local, lo que permite que nuevas instancias se integren sin necesidad de reconfigurar el sistema central.
- ❖ Alta disponibilidad y tolerancia a fallos: Al replicar los datos en múltiples nodos, SBA garantiza que el sistema pueda continuar operando incluso si uno o varios nodos fallan. Esto es esencial para aplicaciones críticas como comercio electrónico, banca o telecomunicaciones.
- ❖ Desacoplamiento funcional: El uso de colas y buses de mensajes para la comunicación entre PUs introduce asincronía, lo que permite que los componentes trabajen de forma independiente y reactiva. Esto mejora la eficiencia y facilita el mantenimiento.

Conceptos centrales:

- ❖ Data Grid / In-Memory Data Store: Los datos se guardan en memoria distribuida (espacio), no en una sola base de datos.

- ❖ **Processing Units (PUs):** Componentes desplegados dinámicamente que contienen lógica de negocio y caché local. Pueden escalar horizontalmente.
- ❖ **Virtualized Middleware:** Maneja la replicación de datos, balanceo de carga y comunicación entre unidades.
- ❖ **Asincronía:** Se usan colas o buses de mensajes para comunicar eventos entre PUs.

El nombre *space-based* viene de la idea de un espacio compartido de datos en memoria, donde todos los nodos pueden leer/escribir de manera distribuida.

Esto reduce la dependencia de la base de datos y permite que el sistema siga funcionando, aunque un nodo falle.

La arquitectura Space-Based fue desarrollado en los años 1980 por David Gelernter en la Universidad de Yale. Este concepto fue tan adelantado a su tiempo que no encontró aplicaciones prácticas inmediatas... hasta que décadas después, con el auge de sistemas distribuidos y la necesidad de escalabilidad extrema.

Diagrama de componentes y flujos

Processing Unit contiene la lógica de la aplicación, incluyendo componentes web y lógica de negocio. Su estructura varía según el tamaño y tipo de aplicación: las más pequeñas pueden desplegarse en una sola unidad, mientras que las más grandes dividen su funcionalidad en varias unidades según áreas específicas. También puede incluir servicios pequeños y especializados, como en arquitecturas de microservicios. Además de la lógica, cada unidad incorpora una cuadrícula de datos en memoria y un motor de replicación.

En otras palabras, es una unidad de ejecución que contiene la lógica de negocio y una caché local. Se encarga de procesar datos, ejecutar tareas y responder a eventos. Puede ser creada o destruida automáticamente según la carga del sistema, lo que permite una adaptación eficiente a picos de tráfico.

Messaging Grid es el sistema de mensajería interna que conecta las diferentes PUs. Propaga eventos y mensajes entre componentes de forma asincrónica, permitiendo una comunicación desacoplada y eficiente. En otras palabras, es el componente encargado de gestionar las solicitudes de entrada y el estado de sesión dentro del middleware virtualizado. Al recibir una solicitud, determina qué unidades de procesamiento están activas y disponibles, y la redirige a una de ellas.

Su complejidad puede variar desde algoritmos simples como *round-robin* hasta otros más avanzados que rastrean qué unidad está procesando cada solicitud.

Generalmente, se implementa mediante servidores web con capacidades de balanceo de carga, como HA Proxy o Nginx.

Deployment Manager es el componente que gestiona el despliegue y la escalabilidad de las PUs. Monitorea la carga del sistema y escala horizontalmente las PUs según la demanda, asegurando que haya suficientes recursos disponibles.

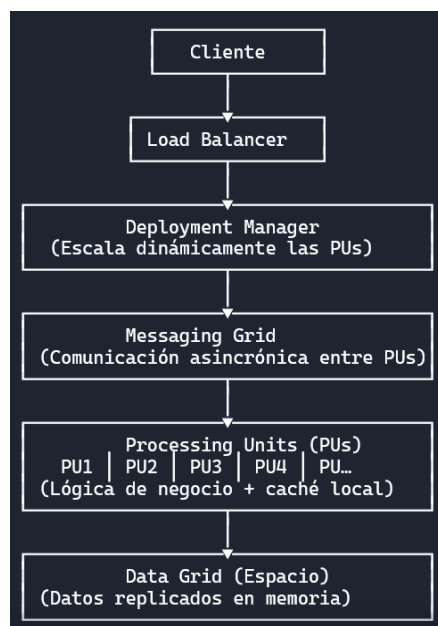
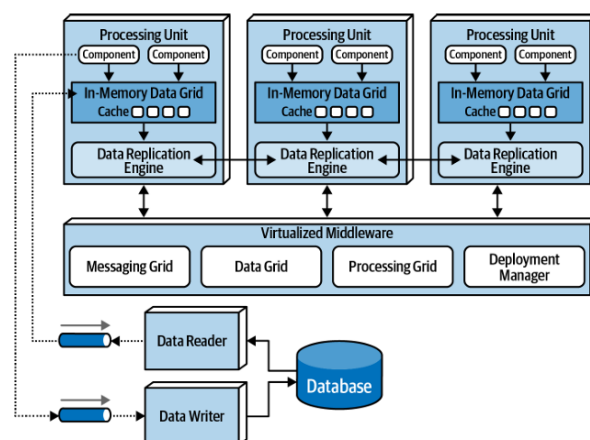


Diagrama lógico de arquitectura Space-Based y flujo de componentes

Este diagrama refleja mejor el desacoplamiento, la escalabilidad dinámica, y la resiliencia que caracterizan a SBA. También muestra cómo los distintos componentes interactúan para mantener el sistema operativo incluso bajo alta carga o fallos parciales.



```

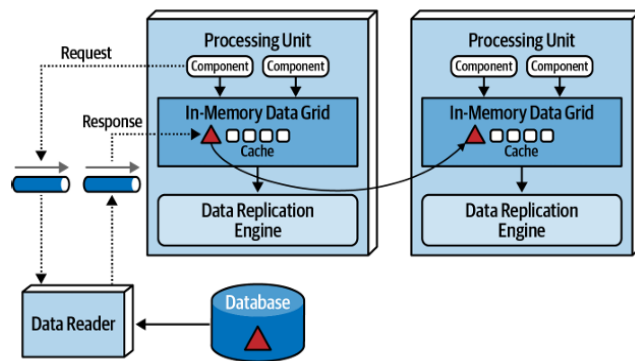
graph TD
    subgraph Processing_Unit [Processing Unit]
        direction TB
        subgraph In_Memory_Data_Grid [In-Memory Data Grid]
            direction TB
            C[Component] --> D[Component]
            D --> IMDG[In-Memory Data Grid]
        end
        IMDG --> Cache[Cache]
        Cache --> DRE[Data Replication Engine]
    end
    Cache --> DW[Data Writer]
    DW --> DB[(Database)]
    DB --> DW
    DW --> Cache
    
```

The diagram shows a 'Data Writer' box on the left and a 'Database' cylinder on the right. An arrow points from the 'Data Writer' to the 'Database'. A curved arrow points from the 'Database' back to the 'Data Writer'. Both the 'Data Writer' and the 'Database' contain a red triangle icon.

...específica (como clientes), sin importar
...aunque haya varias bombas de datos para
...de deseos, Billetera, Preferencias), un solo

escritor de datos puede encargarse de todas ellas, aplicando la lógica necesaria para actualizar la base de datos correctamente.

Data Reader son responsables de recuperar información desde la base de datos y enviarla a las unidades de procesamiento mediante una bomba de datos inversa. Solo se activan en tres casos: cuando todas las instancias de procesamiento



Data reader con data pump inversa

caen, cuando se redeployan, o cuando se necesita acceder a datos archivados que no están en la caché replicada. En estos escenarios, una instancia toma control temporal de la caché, solicita los datos mediante una cola, y el lector de datos realiza la consulta en la base de datos. Luego, los datos se envían a través de una bomba inversa para cargar la caché, tras lo cual se libera el control y se sincronizan las demás instancias para reanudar el procesamiento.

Data Collisions ocurre en arquitecturas activas que usan caché replicada, cuando dos instancias de servicio actualizan simultáneamente el mismo dato en diferentes copias de la caché. Debido a la latencia de replicación, una actualización puede sobrescribirse por otra más antigua, provocando inconsistencias y desincronización entre las instancias.

Un ejemplo típico es el manejo de inventario: si dos servicios actualizan el número de unidades vendidas al mismo tiempo, la replicación cruzada puede hacer que ambos terminen con datos incorrectos, aunque el total real sea distinto.

La probabilidad de colisiones depende de varios factores, número de instancias (que comparten la misma caché), frecuencia de actualizaciones, tamaño de la caché (cantidad de datos) y latencia de replicación.

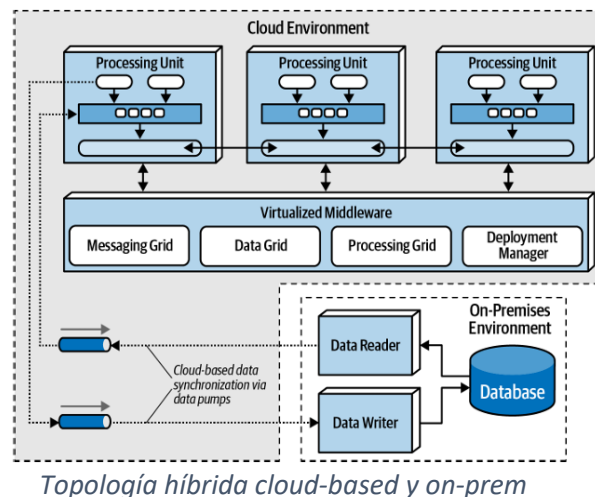
Existe una fórmula para estimar la tasa de colisiones:

$$Collision_Rate = N * \frac{UR^2}{S} * RL$$

Donde:

- **N** = número de instancias de servicio
- **UR** = tasa de actualización (milisegundos, al cuadrado)
- **S** = tamaño de la caché (número de filas)
- **RL** = latencia de replicación

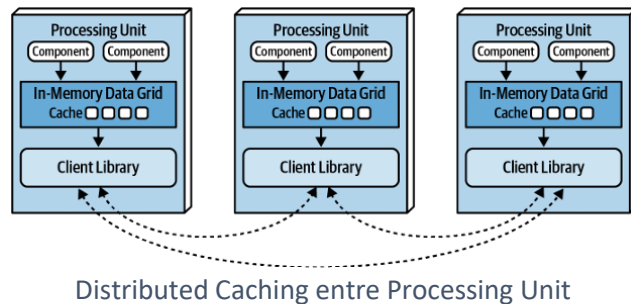
Cloud Versus On-Premises La arquitectura space-based ofrece gran flexibilidad en su implementación, ya que puede desplegarse completamente en la nube, en instalaciones locales (on-premises), o de forma híbrida entre ambos entornos. Esta última opción es una característica distintiva de este estilo arquitectónico.



Una de sus ventajas más poderosas es la posibilidad de ejecutar las aplicaciones (mediante unidades de procesamiento y middleware virtualizado) en entornos gestionados en la nube, mientras se mantienen las bases de datos físicas y los datos sensibles en servidores locales. Esta topología híbrida permite una sincronización eficiente de datos gracias al uso de bombas de datos asincrónicas y al modelo de consistencia eventual.

Así, el procesamiento transaccional se realiza en entornos dinámicos y escalables en la nube, mientras que la gestión de datos, los informes y el análisis permanecen seguros y controlados en instalaciones locales.

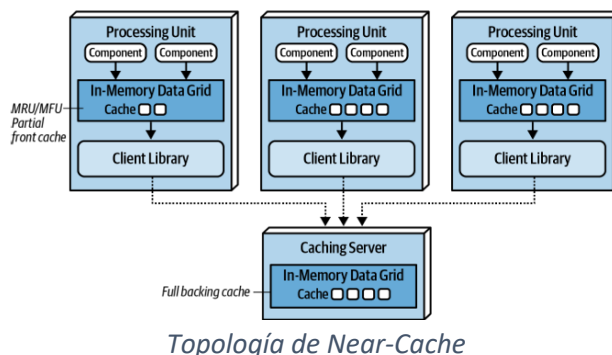
Replicated Versus Distributed Caching La arquitectura space-based utiliza la caché como mecanismo principal para procesar transacciones sin acceder directamente a la base de datos, lo que permite alta escalabilidad, elasticidad y rendimiento. El modelo más común es la caché replicada, donde cada unidad de procesamiento mantiene su propia copia en memoria sincronizada con las demás. Esto ofrece velocidad y tolerancia a fallos, ya que no hay un servidor central que pueda convertirse en un punto único de falla. Sin embargo, algunas implementaciones requieren controladores externos para gestionar la replicación.



A pesar de sus ventajas, la caché replicada puede presentar problemas cuando el volumen de datos es muy alto (más de 100 MB por unidad) o cuando la tasa de actualización es demasiado rápida. En estos casos, la sincronización entre unidades puede fallar, afectando la consistencia de los datos y la escalabilidad del sistema.

Cuando esto ocurre, se puede optar por la caché distribuida, que utiliza un servidor central para almacenar los datos. Las unidades de procesamiento acceden a esta caché mediante protocolos propietarios, lo que garantiza mayor consistencia al tener los datos en un solo lugar. Sin embargo, este modelo tiene menor rendimiento debido a la latencia del acceso remoto, y menor tolerancia a fallos: si el servidor de caché falla, todas las unidades quedan inoperativas. Aunque se puede mitigar con caché espejada, esto introduce riesgos de inconsistencia si el servidor principal cae inesperadamente.

Near-Cache es una solución híbrida que conecta una caché distribuida central (llamada caché de respaldo completa) con cachés locales en memoria dentro de cada unidad de procesamiento (llamadas cachés



frontales). Estas cachés frontales almacenan subconjuntos del respaldo completo y aplican políticas de expulsión como MRU (más recientemente usados), MFU (más frecuentemente usados) o reemplazo aleatorio (RR), dependiendo del comportamiento esperado de los datos. Aunque se sincronizan con la caché central, no se sincronizan entre unidades de procesamiento, lo que genera inconsistencias en el rendimiento y la capacidad de respuesta, ya que cada unidad puede tener datos distintos en su caché local. Por esta razón, no se recomienda el uso del modelo near-cache en arquitecturas space-based, donde la coherencia entre unidades es esencial.

Contextos recomendados para SBA

Como bien se mencionó en contextos anteriores, la arquitectura Sspace-based está diseñada para entornos donde la escalabilidad, la resiliencia y la baja latencia son requisitos críticos. Su enfoque distribuido y en memoria la convierte en una solución ideal para ciertos tipos de sistemas que enfrentan desafíos extremos de rendimiento y disponibilidad.

- ❖ Sistemas con picos de carga extremos, pues en eventos como *Black Friday*, la demanda puede multiplicarse en segundos. Las arquitecturas tradicionales, con bases de datos centralizadas, suelen colapsar bajo esta presión. Aplicamos SBA como solución al distribuir tanto los datos como el procesamiento en memoria, SBA permite que el sistema escale horizontalmente sin saturar un punto único.

Ejemplos:

- Plataformas de comercio electrónico.
- Venta de boletos para conciertos o vuelos.
- ❖ Aplicaciones financieras en tiempo real, pues el *trading de alta frecuencia* y la banca digital requieren procesamiento de transacciones en milisegundos, sin margen para fallos. Aplicamos SBA como solución debido a su capacidad para operar en memoria

y replicar datos entre nodos garantiza velocidad y continuidad, incluso ante fallos parciales.

Ejemplos:

- Sistemas de bolsa.
 - Transferencias bancarias instantáneas.
- ❖ Aplicaciones que requieren disponibilidad continua, pues servicios como *streaming* o *juegos online* no pueden permitirse interrupciones. La experiencia del usuario depende de una disponibilidad constante. Aplicamos SBA como solución al eliminar la dependencia de una base de datos central y permitir que los nodos operen de forma autónoma, SBA asegura continuidad operativa.

Ejemplos:

- Netflix, Twitch, Spotify.
 - Juegos multijugador masivos (MMO).
- ❖ Sistemas sensibles a la latencia, pues en aplicaciones donde cada milisegundo cuenta, como en IoT o control industrial, acceder a una base de datos central puede ser demasiado lento. Aplicamos SBA como solución al mantener los datos en memoria distribuida, se reduce drásticamente la latencia de lectura/escritura.

Ejemplos:

- Sistemas de monitoreo en tiempo real.
- Plataformas de recomendación instantánea.

Análisis Crítico

Ventajas y Desventajas

A continuación, se presenta un análisis técnico que pondera sus principales ventajas y limitaciones. Sobre sus ventajas:

- ❖ **Altísima escalabilidad:** La SBA se basa en PUs que encapsulan tanto la lógica de negocio como la caché local. Estas PUs se despliegan en paralelo y se conectan al Data Grid distribuido. Cuando la carga de trabajo aumenta, basta con añadir más PUs (escalado horizontal). No hay un componente central único (como una base de datos monolítica) que limite la escalabilidad. Cada nueva PU comparte la misma vista de datos gracias al espacio replicado, sin necesidad de rediseñar la aplicación.

Ejemplo: un portal de ventas online puede pasar de 10 a 1000 transacciones por segundo añadiendo más PUs sin rediseñar nada, solo desplegando nodos adicionales.

- ❖ **Baja latencia:** En arquitecturas tradicionales, el cuello de botella suele estar en la base de datos relacional: cada consulta implica operaciones de disco, locking y concurrencia. En SBA, los datos residen en memoria (RAM) distribuida, lo que permite accesos de microsegundos a milisegundos. Esto elimina la latencia asociada a discos o redes de almacenamiento centralizadas. Además, el uso de cachés replicadas locales en cada PU reduce la necesidad de “viajar” a otro servidor para obtener datos.

Ejemplo: en un sistema de trading bursátil, cada milisegundo cuenta. SBA reduce el tiempo de lectura de datos de 10–20 ms (disco) a 0.1–1 ms (memoria).

- ❖ **Alta disponibilidad:** La tolerancia a fallos en SBA se logra por la replicación de datos en múltiples nodos. Si un nodo o una PU se cae, otras pueden seguir operando con una copia consistente de los datos. La replicación puede ser sincrónica (cada escritura se refleja inmediatamente en varias réplicas) o asincrónica (más rápida, pero con riesgo de pequeños retrasos). Esto significa que el sistema no depende de un único servidor, eliminando el “punto único de fallo”.

Ejemplo: en un sistema de reservas aéreas, si un servidor se desconecta, otro nodo con la misma información sigue respondiendo y el cliente no percibe la caída.

- ❖ Elasticidad: La elasticidad es la capacidad de ajustar los recursos en tiempo real según la demanda. SBA permite activar o desactivar PUs dinámicamente de acuerdo con métricas de carga (CPU, peticiones por segundo, etc.). Esto es posible gracias a que las PUs son componentes autónomos y ligeros, fácilmente desplegables en contenedores o máquinas virtuales. La infraestructura puede integrarse con orquestadores como Kubernetes, que escalan automáticamente el número de PUs según la demanda.

Ejemplo: durante un Black Friday, el sistema activa 50 PUs adicionales para procesar compras; al terminar la jornada, las desactiva para ahorrar recursos.

Sobre sus desventajas:

- ❖ Complejidad de diseño y mantenimiento: SBA es una arquitectura distribuida, lo que implica múltiples nodos, sincronización de datos, manejo de fallos, monitoreo continuo y despliegue dinámico. Estos elementos requieren una planificación meticulosa y una infraestructura robusta.
- ❖ Consistencia de datos: Al operar sobre un *Data Grid* en memoria distribuida, mantener la coherencia entre nodos es un reto. Estrategias como *eventual consistency*, *locking* distribuido o *conflict resolution* deben implementarse cuidadosamente.
- ❖ Curva de aprendizaje: SBA introduce conceptos avanzados como procesamiento distribuido, replicación en memoria, escalado dinámico y comunicación asincrónica. Para equipos sin experiencia en sistemas distribuidos, esto puede representar una barrera significativa.
- ❖ Sobredimensionamiento en aplicaciones pequeñas: No todas las aplicaciones necesitan la potencia de SBA. Para sistemas con baja carga, tráfico predecible o lógica sencilla, esta arquitectura puede resultar innecesariamente compleja.

Impacto en calidad de software

Rendimiento: El impacto más evidente de la arquitectura *Space-Based* en la calidad del software se refleja en el rendimiento. Al sustituir la base de datos central por un Data Grid en memoria distribuida, se elimina el principal cuello de botella en sistemas tradicionales. Esto permite tiempos de respuesta mucho menores (en el orden de microsegundos a pocos milisegundos) y una mayor capacidad para atender altos volúmenes de transacciones concurrentes. Además, el escalado horizontal mediante *Processing Units* posibilita que el sistema aumente su throughput sin degradar la experiencia del usuario, ya que cada nueva PU comparte la misma vista de los datos. En consecuencia, la SBA es especialmente eficaz en aplicaciones que requieren baja latencia y un rendimiento estable bajo picos de carga extremos, como sistemas financieros o comercio electrónico masivo.

Seguridad: En cuanto a la seguridad, la SBA plantea un escenario particular. Al tratarse de un sistema distribuido en memoria, los datos residen en múltiples nodos, lo cual aumenta la superficie de ataque en comparación con una arquitectura centralizada. Es necesario implementar mecanismos de encriptación de datos en tránsito y en reposo, así como políticas de autenticación y autorización que controlen el acceso al Data Grid. Sin embargo, la propia descentralización también aporta ventajas: al no existir un único punto de almacenamiento, es más difícil que un ataque comprometa la totalidad de la información. En este sentido, la seguridad en SBA depende fuertemente de una infraestructura bien diseñada, con monitoreo continuo, firewalls distribuidos y protocolos de replicación seguros, de manera que el aumento en escalabilidad y disponibilidad no se traduzca en vulnerabilidades.

Mantenibilidad: Respecto a la mantenibilidad, la SBA introduce tanto ventajas como desafíos. Por un lado, el hecho de que las *Processing Units* sean componentes autónomos y replicables facilita la evolución y actualización del sistema: se pueden desplegar nuevas versiones de una PU sin interrumpir el servicio, siempre que se mantenga la compatibilidad con el Data Grid. Esto promueve la modularidad y la

independencia de componentes, favoreciendo el mantenimiento incremental. No obstante, la complejidad de coordinar múltiples nodos distribuidos puede dificultar las tareas de depuración, monitoreo y sincronización de datos. A diferencia de una arquitectura monolítica, donde los errores suelen localizarse en un único entorno, en SBA se requiere instrumentación avanzada, trazabilidad de eventos y un fuerte soporte de automatización para asegurar que los cambios locales no generen inconsistencias globales. En conclusión, la SBA es mantenible si el equipo de desarrollo cuenta con buenas prácticas de ingeniería de software distribuido, pero puede resultar costosa en entornos con baja madurez técnica.

Comparación

Característica	Space-Based	Microservices	Event-Driven
Escalabilidad	Muy alta (escalado horizontal en memoria)	Alta (servicios independientes)	Alta (procesamiento reactivo por eventos)
Persistencia	En memoria distribuida (Data Grid)	Base de datos por servicio	Event Store o colas persistentes
Complejidad	Alta (sincronización y replicación)	Media-Alta (gestión de múltiples servicios)	Alta (orquestración de eventos y consistencia)
Latencia	Muy baja (acceso directo en memoria)	Variable (depende del acceso a BD)	Variable (depende del bus de eventos)
Elasticidad	Dinámica (creación/destrucción de PUs)	Escalable por servicio	Escalable por consumidores de eventos
Uso típico	E-commerce masivo, trading, streaming	Sistemas modulares, plataformas SaaS	IoT, notificaciones, sistemas reactivos

Microservices se popularizaron gracias a empresas como Netflix y Amazon en la década de 2010.

Event-Driven Architecture tiene raíces en los sistemas de mensajería de los años 80 y 90, como IBM MQSeries.

Bibliografía

- ❖ Richards, M. & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.
- ❖ Pautasso, C., Zimmermann, O., & Leymann, F. (2017). *Architectural Styles for Distributed Systems*. Springer.
- ❖ GigaSpaces Technologies. (2021). *Space-Based Architecture Overview*.